

## Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

# 1. Introduction

This report documents Phase 3 of my Personal Research Portal, which extends the research-grade RAG pipeline built in Phase 2 into an interactive portal for exploring machine learning failure modes in space debris tracking and collision avoidance. Where Phase 2 focused on retrieval quality, citation traceability, and evaluation rigor, Phase 3 wraps those capabilities into a usable research workflow: question → evidence → synthesis → export.

The portal addresses three goals. First, it provides an interface for querying the 20-source corpus with metadata filtering by year, source type, and domain-specific tags, surfacing relevant evidence with full citation traceability. Second, it generates research artifacts (structured evidence tables and synthesis memos) from saved research threads, each grounded in the same `(source_id, chunk_id)` citation system established in Phase 2. Third, it incorporates the Phase 2 evaluation results into an integrated dashboard, making the system's strengths and limitations transparent to the user.

The system also addresses Phase 2 feedback on robustness and scalability by implementing a cache-based offline mode that allows the full application to function without an API key, graceful degradation under API failures, and rate limiting to prevent accidental overuse.

# 2. System Architecture

The Phase 3 portal builds on the Phase 2 pipeline without modifying its core behavior. The existing retrieval, reranking, generation, and logging modules are preserved exactly as evaluated. The Phase 2 evaluation results remain reproducible. Phase 3 adds four new module groups: the Streamlit application layer, a caching system, a thread manager, and an artifact generator.

## 2.1 Pipeline Integration

The Phase 2 pipeline is exposed to the portal through a `RAGPipeline` class added alongside the existing `run_query()` function in `src/rag/pipeline.py`. The class holds the ChromaDB collection and reranker model in instance state, initialized once via Streamlit's `@st.cache_resource` decorator to avoid reloading models on every page interaction. The class exposes two methods: `run()` for full pipeline execution (retrieve → rerank → generate → log) and `retrieve_only()` for offline fallback where generation is unavailable but local retrieval and reranking still work.

The critical interface addition is the `where` parameter, which accepts a ChromaDB metadata filter dict and passes it through to the retriever. This enables the Phase 3 metadata filtering enhancement without changing the retrieval logic itself.

## 2.2 Metadata Filtering Enhancement

Metadata filtering is the designated Phase 3 pipeline enhancement. During Phase 2, the corpus tags were added late and had significant problems: the `machine-learning` tag appeared on 19 of 20 sources (providing zero filtering value),

Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

source type tags duplicated the existing `doc_type` metadata field, and the failure-mode dimension—central to the research question—was entirely absent.

For Phase 3, the tag scheme was redesigned from scratch around three orthogonal axes:

**Domain Application** captures what area of space debris the source addresses: collision-avoidance, orbit-prediction, debris-environment, or detection-classification. These are the broadest topical categories.

**ML Limitation Theme** maps directly to the research sub-questions: data-quality (sub-Q2), generalization (sub-Q3), uncertainty-quantification (sub-Q4), and operational-integration (sub-Q5/Q6). This is the dimension most aligned with the research question and the one most useful for a researcher filtering evidence.

**Content Role** describes how the paper functions in the corpus: survey, benchmark, or policy. This helps users find review papers versus original research versus institutional guidance.

The resulting 11 tags each appear on 2–7 sources (average 2.5 tags per source), meaning every filter selection meaningfully narrows the result set. Tags are stored as individual boolean metadata fields in ChromaDB, one field per tag (e.g., `tag_operational_integration: True`). and queried via native `$eq` operators that compose with `$and` for multi-tag selections. The Phase 2 implementation stored tags as semicolon-separated strings and relied on substring matching, which silently returned empty results for tags that never appeared alone. The boolean field approach was adopted after this failure surfaced during cache seeding, and it scales cleanly to additional tags without requiring enumeration of all possible tag combinations. The full corpus was re-ingested with the new tag metadata; all other metadata and embeddings remain identical to Phase 2.

| Axis                | Tags                                                                                | Sources  |
|---------------------|-------------------------------------------------------------------------------------|----------|
| Domain Application  | collision-avoidance, orbit-prediction, debris-environment, detection-classification | 4–7 each |
| ML Limitation Theme | data-quality, generalization, uncertainty-quantification, operational-integration   | 5–7 each |
| Content Role        | survey, benchmark, policy                                                           | 2–4 each |

2.3 Cache and Offline Mode

Phase 2 feedback identified robustness under failures as an area for improvement. The Phase 3 cache system addresses this directly by providing transparent, two-level caching of all LLM-generated content.

Level 1 (query cache) stores the full result of each pipeline run—answer text, retrieved chunks, reranked chunks, model name, and token usage—keyed by a deterministic hash of the query text, reranker flag, and filter parameters. Level 2 (artifact cache) stores generated evidence tables and synthesis memos, keyed by thread ID and artifact type. Both levels use JSON files under `data/cache/`, making cached results inspectable and version-controllable.

## Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

Mode detection is automatic: if `ANTHROPIC_API_KEY` is set in the environment, the app runs in live mode, caching every result as a side effect. Without the key, the app runs in offline mode, serving cached results and falling back to `retrieve_only()` for uncached queries. A banner indicates which mode is active. This means the grader can run the full application without providing an API key, exploring pre-seeded queries and pre-generated artifacts from the cache.

All API calls are wrapped with structured error handling: `RateLimitError` triggers a warning and cache fallback, `APIConnectionError` shows a connection message with cache fallback, and generic exceptions are caught and displayed without crashing the application. A minimum 2-second interval between API calls provides basic rate limiting.

### 2.4 Application Structure

The portal is a multi-page Streamlit application launched via `streamlit run src/app/Ask_Questions.py`. The main page (Ask Questions) provides the query interface with metadata filters, citation display, and thread saving. Three additional pages provide thread management (Threads), the evaluation dashboard (Evaluation), and corpus and architecture documentation (About). The application adds six new modules to the Phase 2 codebase:

| Module                                  | Purpose                                                               |
|-----------------------------------------|-----------------------------------------------------------------------|
| <code>src/cache/manager.py</code>       | Two-level response cache with deterministic hashing                   |
| <code>src/threads/manager.py</code>     | Save, load, list, and delete research threads (JSON)                  |
| <code>src/artifacts/generator.py</code> | Evidence table and synthesis memo generation with citation validation |
| <code>src/artifacts/prompts.py</code>   | Artifact prompt templates (evidence table, synthesis memo)            |
| <code>src/app/components/</code>        | Reusable UI components: citation cards, filter sidebar, mode banner   |
| <code>scripts/seed_cache.py</code>      | Pre-submission cache seeding for offline demo                         |

## 3. Portal Design

The portal is organized as four pages, each supporting a distinct stage of the research workflow. The Ask Questions page handles querying and evidence gathering. The Threads page manages saved research sessions and artifact generation. The Evaluation page provides transparency into system performance. The About page documents the corpus, architecture, and citation system.

### 3.1 Ask Questions Page

The Ask Questions page is the primary research interface. The sidebar presents metadata filters—year range (slider), source type (multiselect), and three tag axes (domain, ML limitation, content role)—which compose into a ChromaDB `$and` clause passed through the `where` parameter at retrieval time. Selecting no filters searches the full corpus. The main area contains a query input field and, after submission, the generated answer with inline citations, expandable citation cards for each referenced chunk, and an evaluation gaps section.

## Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

Citation cards display the chunk's source ID, chunk ID, year, document type, tags, venue, and reranker relevance score, with the full chunk text available via an expander. Citations in the answer text are bolded only when they match real retrieved chunks, a visual trust cue that avoids lending false confidence in the event of a hallucinated reference. After reviewing the answer, a "Save to Thread" button at the bottom persists the full result (query, answer, retrieved and reranked chunks, gap suggestions) as a research thread.

The gap finder is integrated into live answer generation via a suffix appended to the system prompt. After the main answer, the LLM identifies 2–3 specific evidence gaps and suggests follow-up queries for each. These are displayed in a collapsible expander below the citation cards. Each suggested query has a "Search this" button that prefills the query input and runs the search, creating a natural research workflow: ask → read → identify gaps → follow up. Gap suggestions are cached as part of the query result, so they appear in offline mode as well.

### 3.2 Research Threads

Research threads persist the full state of a query session: the query text, filter parameters, generated answer, all retrieved and reranked chunks with metadata, gap suggestions, and any generated artifacts. Threads are stored as JSON files under `outputs/threads/` with timestamped IDs (`thr_YYYYMMDD_HHMMSS_{random}`). The Threads page provides a selectbox to browse saved threads, displaying the full answer with citation cards and offering artifact generation buttons.

From a loaded thread, the user can generate an evidence table or synthesis memo. Both artifact types are cached after generation, so subsequent visits to the same thread serve the artifact immediately. Artifacts display inline on the Threads page. The evidence table is presented as a sortable dataframe; the synthesis memo is rendered as Markdown. There are buttons for CSV, Markdown, and PDF export. Threads can be deleted individually.

### 3.3 Evaluation Dashboard

The Evaluation page presents the Phase 2 evaluation results as an interactive dashboard, fully offline by default. The top row displays five summary metrics as `st.metric` widgets showing reranking-mode scores with deltas against the baseline: groundedness (4.00,  $\pm 0.00$ ), citation correctness (3.96,  $-0.04$ ), completeness (3.72,  $+0.04$ ), retrieval recall (0.84,  $\pm 0.00$ ), and context utilization (0.46,  $+0.06$ ). The deltas make the reranking enhancement's impact visible at a glance.

Two Plotly charts follow. A grouped bar chart compares reranked versus baseline scores across the three query categories (direct, synthesis, edge-case), with a metric selector dropdown. A radar chart normalizes all five metrics to a 0–1 scale and overlays the reranked and baseline profiles, providing a high-level visual comparison. Below the charts, a sortable detail table lists all 50 runs with filtering by category and mode. Each row can be expanded to show the full query text, answer excerpt, judge rationale, and any failure tags.

An insights section summarizes the key findings: perfect groundedness across both modes, the +15% context utilization improvement from reranking (concentrated in synthesis queries), completeness as the weakest and most discriminating metric, and the reranking–diversity tension visible in edge-case queries.

Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

4. Research Artifacts

The portal generates two artifact types from saved research threads: evidence tables and synthesis memos. Both are grounded in the same (source\_id, chunk\_id) citation system used throughout the pipeline, and both undergo citation validation before display.

4.1 Evidence Table

The evidence table extracts discrete claims from a thread’s answer and maps each to its supporting evidence, citation, confidence level, and any caveats. The generator receives the thread’s reranked chunks and answer text, then produces a structured JSON response via Claude Sonnet with a purpose-built prompt. Each row captures one claim with five fields: the claim statement, supporting evidence text, the (source\_id, chunk\_id) citation, a confidence rating (high, medium, or low), and notes for caveats or conflicting evidence.

A key design decision was the handling of evidence gaps. When the pipeline’s answer explicitly flags missing evidence (as it does for queries where retrieval missed expected sources) the evidence table captures these as low-confidence rows rather than silently omitting them. This preserves the trust behavior established in Phase 2, where the system was designed to honestly acknowledge what it does not know. During testing, the initial validation logic incorrectly flagged these gap rows as errors (since they lack a resolvable chunk citation). The fix was to exempt rows with a confidence rating of “low” and a notes field indicating missing evidence from the strict citation check.

A typical evidence table for a synthesis query contains 8–12 rows, with most claims rated high confidence and 1–3 gap rows rated low. The table exports as CSV for use in spreadsheet tools or as input to further analysis.

| claim                                                                                                                                                                                                                             | evidence                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | source_id            | chunk_id    | confidence |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------|------------|
| Data scarcity is a fundamental challenge with the operational use of AI/ML for conjunction risk assessment.                                                                                                                       | The study identified fundamental challenges with the operational use of AI/ML for conjunction risk assessment, including data scarcity, the stochastic nature of orbital mechanics, model interpretability requirements, and the critical need for explainable AI approaches that can meet the high-reliability standards essential for space operations decision-making.                                                                                                                                                                           | mashiku2025          | sec0_p1     | high       |
| Approximately five-sixths of objects large enough to disable satellites are untracked, meaning greater than 85% of similarly likely collisions cannot be addressed at all.                                                        | This means that, roughly speaking, about five-sixths of the objects large enough to leave a satellite in a failed and uncontrolled state if a collision should occur with them will be untracked. Given then that only about one-sixth of the close-approach events are trackable and in principle actionable, the overall conjunction risk analysis process should be commensurate with a situation in which greater than 85% of similarly likely collisions cannot be addressed at all and thus constitute a risk that simply has to be accepted. | nasa_ca_handbook2023 | sec15.1_p3  | high       |
| Model interpretability requirements and the need for explainable AI approaches that can meet high-reliability standards are critical barriers to operational use of AI/ML.                                                        | The study identified fundamental challenges with the operational use of AI/ML for conjunction risk assessment, including data scarcity, the stochastic nature of orbital mechanics, model interpretability requirements, and the critical need for explainable AI approaches that can meet the high-reliability standards essential for space operations decision-making.                                                                                                                                                                           | mashiku2025          | sec0_p1     | high       |
| The stochastic nature of orbital mechanics is a fundamental challenge for operational use of AI/ML in conjunction risk assessment.                                                                                                | The study identified fundamental challenges with the operational use of AI/ML for conjunction risk assessment, including data scarcity, the stochastic nature of orbital mechanics, model interpretability requirements, and the critical need for explainable AI approaches that can meet the high-reliability standards essential for space operations decision-making.                                                                                                                                                                           | mashiku2025          | sec0_p1     | high       |
| Fully automated AI/ML decision systems are not currently viable for operational risk assessment.                                                                                                                                  | While fully automated AI/ML decision systems are not currently viable for operational risk assessment, hybrid approaches combining traditional physics-based methods with targeted machine learning components may offer incremental improvements.                                                                                                                                                                                                                                                                                                  | mashiku2025          | sec6_p3     | high       |
| Hybrid approaches combining traditional physics-based methods with targeted machine learning components may offer incremental improvements.                                                                                       | While fully automated AI/ML decision systems are not currently viable for operational risk assessment, hybrid approaches combining traditional physics-based methods with targeted machine learning components may offer incremental improvements.                                                                                                                                                                                                                                                                                                  | mashiku2025          | sec6_p3     | high       |
| Space domain awareness operators face far more demand on the SSN than availability and must coordinate observation collection according to sometimes conflicting data needs, priorities, and sensor suitability and availability. | Even under normal operating conditions, there is far more demand on the SSN than availability. SDA tasking must account for the relative priority of observation, so analysts must coordinate observation collection according to sometimes conflicting data needs, priorities, and sensor suitability and availability.                                                                                                                                                                                                                            | rand_ai_ssa2024      | sec1.1.3_p2 | high       |

Figure 1. Evidence table exported as CSV from seed thread #3 (operational-integration tag filter). Each row maps a discrete claim to its supporting chunk text, (source\_id, chunk\_id) citation, and confidence rating. Rows 1, 3, and 4 share the same source chunk (mashiku2025, sec0\_p1) but extract distinct claims, demonstrating how the generator decomposes a dense paragraph into individually verifiable assertions.

Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

4.2 Synthesis Memo

The synthesis memo produces an 800–1,200 word research document from a thread’s evidence chunks. Unlike the evidence table, the memo receives only the chunks, not the pipeline’s answer, so that it produces an independent synthesis rather than a restatement. This was a deliberate design choice: if the memo simply rewrote the answer, it would add no analytical value. By synthesizing directly from the evidence, the memo may organize themes differently, surface connections the pipeline’s answer missed, or emphasize different aspects of the evidence.

The memo prompt enforces thematic organization (not source-by-source summarization), inline citations in (source\_id, chunk\_id) format, and an explicit treatment of evidence gaps. The output is validated for word count (target 800–1,200), citation count (target 10–20), and citation validity against the thread’s available chunks. Memos export as Markdown and PDF via fpdf2.

PDF generation required handling non-Unicode characters from corpus chunks—em-dashes, curly quotes, and special characters from aerospace terminology. The solution was to use a Unicode-compatible font and sanitize characters before rendering. This was a practical issue discovered during testing that would not have surfaced from architecture planning alone.



Figure 2. Example synthesis memo output, generated from a thread filtered by the operational-integration tag. The memo demonstrates thematic organization (not source-by-source), inline (source\_id, chunk\_id) citations, and an executive summary, all produced from the evidence chunks alone, without access to the pipeline's original answer.

Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

4.3 Citation Validation

Both artifact types pass through a citation validation step before display. Every (source\_id, chunk\_id) pair in the generated output is checked against the thread’s available chunks. Invalid citations (those referencing chunks not in the thread’s evidence) trigger a visible warning. This mechanical check complements the prompt-level instruction not to fabricate citations, providing a defense-in-depth approach to trust behavior.

5. Evaluation and Trust

The evaluation dashboard integrates Phase 2 results directly into the portal, making the system’s performance characteristics visible to users. The dashboard is fully offline. It reads from logs/eval\_results\_v2.jsonl (50 records) and logs/eval\_results\_v2\_summary.json, requiring no API calls.

5.1 Phase 2 Results Summary

The Phase 2 evaluation ran 25 queries (12 direct, 7 synthesis, 6 edge-case) in two modes (with and without reranking) for 50 total runs, scored by Claude Opus 4.6 as an LLM judge. All 50 runs completed without errors.

With Reranking:

| Category  | n  | Grounded. | Citation | Complete. | Ret. Recall | Ctx. Util. |
|-----------|----|-----------|----------|-----------|-------------|------------|
| Direct    | 12 | 4.00      | 4.00     | 3.75      | 0.88        | 0.38       |
| Synthesis | 7  | 4.00      | 4.00     | 3.57      | 0.89        | 0.64       |
| Edge-case | 6  | 4.00      | 3.83     | 3.83      | 0.67        | 0.38       |
| Overall   | 25 | 4.00      | 3.96     | 3.72      | 0.84        | 0.46       |

Baseline (No Reranking):

| Category  | n  | Grounded. | Citation | Complete. | Ret. Recall | Ctx. Util. |
|-----------|----|-----------|----------|-----------|-------------|------------|
| Direct    | 12 | 4.00      | 4.00     | 3.75      | 0.88        | 0.30       |
| Synthesis | 7  | 4.00      | 4.00     | 3.57      | 0.89        | 0.57       |
| Edge-case | 6  | 4.00      | 4.00     | 3.67      | 0.67        | 0.40       |
| Overall   | 25 | 4.00      | 4.00     | 3.68      | 0.84        | 0.40       |

The dashboard presents these results through interactive Plotly visualizations: a grouped bar chart comparing metrics across query categories and modes, and a radar chart providing a high-level comparison of reranked versus baseline performance. A sortable detail table allows inspection of individual query results, including the full answer text and retrieved chunks.

5.2 Trust Behaviors

Trust behavior is a requirement that spans all three phases. In Phase 1, prompt engineering established the citation format and the instruction to refuse fabrication. In Phase 2, the pipeline implemented this through a structured prompt



## Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

(v1.0) that enforces `(source_id, chunk_id)` citations and explicitly flags insufficient evidence. Phase 3 extends trust behavior in three ways.

**Citation traceability in artifacts.** Both artifact types preserve the same citation format used in pipeline answers. The evidence table maps every claim to a specific chunk, and the synthesis memo uses inline citations throughout. Citation validation checks that every reference resolves to a real chunk in the thread's evidence.

**Gap identification.** The gap finder extends the generation prompt to identify missing evidence and suggest follow-up queries. When the system detects that retrieved evidence does not fully cover the question, as occurred with S04 in Phase 2, where RAND sources were absent from retrieval, it explicitly states what is missing and proposes specific queries to fill the gap. These suggestions appear as clickable buttons in the UI, creating a natural research workflow.

**Transparent evaluation.** Integrating the evaluation dashboard into the portal itself, rather than presenting it only in the report, makes the system's performance characteristics accessible to any user. The dashboard shows not just aggregate scores but per-query details, including failure cases and the specific failure tags assigned by the judge (MISSED\_EVIDENCE, FABRICATED\_CITATION, etc.).

### 5.3 Metadata Filtering Impact

Metadata filtering addresses a specific failure mode identified in Phase 2: vocabulary mismatch between queries and corpus chunks. The S04 query ("common reasons cited across NASA, ESA, and RAND sources") missed both mashiku2025 and rand\_ai\_ssa2024 because the query's organizational names did not match the chunks' technical vocabulary. With metadata filtering, a user can constrain retrieval to sources tagged `operational-integration`, which includes both mashiku2025 and rand\_ai\_ssa2024, directly surfacing the evidence that pure vector search missed.

Tags are stored as individual boolean metadata fields in ChromaDB (e.g., `tag_operational_integration: True`), queried via native `$eq` operators. This approach avoids the fragility of substring matching on compound strings and composes cleanly with `$and` for multi-tag queries. The seed cache includes a query filtered by the `operational-integration` tag, which successfully retrieved chunks from mashiku2025, rand\_ai\_ssa2024, nasa\_ca\_handbook2023, and acciarini2021, producing a 10-claim evidence table where all claims traced to directly relevant source material (Figures 1–2).

This is a manual workaround rather than an automated fix, since the user must know which tags to select. However, it transforms an invisible retrieval failure into a recoverable one. The gap finder provides an additional bridge: when a query's results are incomplete, the suggested follow-up queries may benefit from tag filtering, guiding the user toward evidence the initial search missed.



## Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

### 6. Limitations and Scalability

Several limitations identified in Phase 2 persist into Phase 3, and the portal's expanded functionality introduces new considerations.

#### 6.1 Retrieval Vocabulary Mismatch

The most significant pipeline limitation remains vocabulary mismatch between queries and corpus chunks. Phase 2 documented this clearly: queries using organizational names or abstract concepts fail to retrieve chunks that use domain-specific technical vocabulary. Retrieval recall of 0.84 overall (0.67 for edge cases) confirms this is systematic. Phase 3's metadata filtering partially addresses this: a user can constrain retrieval to sources tagged with `operational-integration` to surface chunks from `mashiku2025` and `rand_ai_ssa2024` that were missed by pure vector search in S04. The boolean metadata field approach (one field per tag, queried via native `$eq` operators) ensures this filtering is reliable and composable. However, filtering remains a manual intervention that requires the user to know which tags to select, not a fix to the embedding mismatch itself. Query decomposition or hybrid retrieval (BM25 + vector) would address the root cause by resolving vocabulary mismatches automatically at retrieval time.

#### 6.2 Chunking Scalability

All 20 papers were chunked manually with Claude following a structured protocol. This produced high-quality, semantically coherent chunks with accurate section boundaries. This worked significantly better than automated OCR extraction, which was attempted and rejected during Phase 2 due to poor accuracy on mathematical notation, tables, and multi-column layouts. For 20 papers, this was the correct decision. For a larger corpus (100+ papers), it would not scale.

A practical scaling path would be LLM-assisted chunking with spot-checks: use Claude or a similar model to produce initial chunks from PDF text, then have a human reviewer verify a random sample (e.g., 10–20%) and correct systematic errors. Alternatively, structured PDF extraction tools (e.g., GROBID for academic papers) with OCR fallback could handle well-formatted papers, reserving manual chunking for sources with complex layouts.

#### 6.3 API Dependency

The portal depends on the Anthropic API for answer generation and artifact creation. The cache system mitigates this for demo and grading purposes. The pre-seeded cache provides a rich offline experience, but live usage requires a valid API key and stable internet connection. The rate limiting and error handling added in Phase 3 make this dependency more graceful, but they do not eliminate it. A production system should consider local model deployment or multi-provider fallback.

#### 6.4 Synthesis Completeness

The completeness gap on synthesis queries (3.57/4.00, the lowest score in the evaluation) suggests the generator anchors on a few strong sources rather than integrating all available evidence. A potential improvement is a generation prompt that explicitly instructs the model to use every retrieved chunk, or a post-generation check that identifies cited

## Machine Learning Failure Modes in Space Debris Tracking & Collision Avoidance

versus uncited chunks and prompts a second pass to incorporate unused evidence. Alternatively, the gap finder partially addresses this by surfacing missing dimensions that the user can pursue with follow-up queries.

## 7. Conclusion

Phase 3 transforms the Phase 2 RAG pipeline into a functional research portal that supports the full research workflow: question, evidence gathering, synthesis, and export. The portal's metadata filtering, built on a redesigned tag scheme with 11 tags across three axes, gives researchers meaningful control over which evidence the system considers. The cache-based offline mode, error handling, and rate limiting address the robustness concerns raised in Phase 2 feedback.

The artifact generation system (evidence tables and synthesis memos) produces research deliverables grounded in the same citation infrastructure that underpins the pipeline itself. Citation validation ensures that the trust behavior established in Phase 1 and enforced in Phase 2 extends through to the final research outputs.

The evaluation dashboard's integration into the portal, rather than existing only as a separate script, reflects a broader design principle: a research tool should be transparent about its own capabilities and limitations. By surfacing retrieval misses, judge calibration data, and per-query failure analysis alongside the research interface, the portal helps users make informed judgments about the evidence they receive.